

API

El primer paso del trabajo práctico consistió en el desarrollo de una API utilizando Spring Boot, la misma cuenta con distintos endpoints para exponer diferentes funcionalidades.

Además, se desarrollaron tests automatizados para validar el comportamiento poder validar el correcto funcionamiento de la aplicación

DOCKER / DOCKER-COMPOSE

La aplicación fue dockerizada, implementando un enfoque de build en múltiples etapas (multi-stage build).

En la primera etapa (builder) se utiliza una imagen de Maven para compilar el proyecto. Se descargan las dependencias y se genera el archivo .jar de la aplicación. Esto permite separar el proceso de build del entorno final de ejecución.

En la segunda etapa, se utiliza una imagen más liviana de Java (Eclipse Temurin) donde únicamente se copia el .jar. Esto reduce el tamaño de la imagen final y mejora la eficiencia.

Además, se incorpora el agente de New Relic, lo que permite agregar monitoreo de la aplicación en tiempo de ejecución.

Por otro lado se utilizó Docker Compose para definir y levantar el servicio. Se configuraron variables de entorno mediante un archivo .env, y se agregó un healthcheck que consulta el endpoint /health para verificar el estado de la aplicación.

GIT / GITHUB

El código del proyecto se gestionó utilizando Git y se alojó en GitHub.

Se protegió la rama main para evitar pushes directos, obligando a que todos los cambios se integren mediante Pull Requests. Cada Pull Request dispara automáticamente el pipeline de CI, el cual ejecuta tests y validaciones sobre el código. Solo se permite hacer merge cuando el pipeline finaliza correctamente, garantizando que ningún cambio rompa la aplicación ni el proceso de build.

Una vez que los cambios son mergeados a main, se ejecuta automáticamente el proceso de CD, permitiendo desplegar la aplicación de forma continua.

CI / CD

Se implementaron pipelines de CI/CD utilizando GitHub Actions para automatizar el proceso de validación y despliegue de la aplicación.

El pipeline de CI se ejecuta automáticamente ante cada push o Pull Request hacia la rama main.

Durante este proceso:

- Se configura el entorno con Java 21
- Se inyectan variables de entorno necesarias (base de datos)
- Se compila la aplicación y se ejecutan los tests (mvn clean test)

Esto permite validar automáticamente que el código funcione correctamente antes de ser integrado.

El pipeline de CD se ejecuta únicamente cuando el workflow de CI finaliza correctamente y los cambios provienen de la rama main.

Durante el proceso de CD:

- Se construye una imagen Docker de la aplicación
- La imagen se etiqueta tanto con el hash del commit como con latest
- Se realiza login y push de la imagen a Docker Hub
- Se dispara un deploy en Render utilizando un hook, pasando la versión específica de la imagen

DESPLIEGUE

Para el despliegue de la aplicación y las dos bases de datos (producción y testing) se utilizó Render, que permite automatizar la publicación de servicios a partir de imágenes docker.

El proceso de deploy está integrado con el pipeline de CD. Una vez que se genera y publica la imagen en Docker Hub, se utiliza un deploy hook de Render para actualizar el servicio automáticamente.

En cada despliegue:

- Se envía la URL de la imagen específica (taggeada con el hash del commit)
- Render descarga esa versión exacta y la despliega
- Se evita depender de la etiqueta latest, asegurando mayor control y trazabilidad

OBSERVABILIDAD

Se integró New Relic para monitorear el comportamiento de la aplicación en tiempo de ejecución.

La integración se realizó mediante un agente Java incluido dentro de la imagen Docker, el cual se ejecuta junto con la aplicación utilizando la opción -javaagent. Esto permite recolectar métricas sin necesidad de modificar el código fuente.

Conceptos clave:

Three Ways

Son tres principios para mejorar cómo se desarrolla y entrega software:

Flujo en el TP: lo logramos con CI/CD, Docker y deploy automático en Render. Cada cambio en main termina desplegado sin pasos manuales.

Feedback loops en el TP: usamos tests en el pipeline, PRs obligatorios y New Relic para monitorear. Así sabemos rápido si algo falla.

Continuous Learning en el TP: con métricas de New Relic y versiones por commit podemos analizar errores y mejorar la app.

Andon Cords

Es la idea de frenar todo cuando hay un error, para no seguir propagándolo.

En el TP:

Si fallan los tests → no se puede hacer merge

Si falla CI → no se ejecuta CD

Metodologías Ágiles

Se basan en trabajar de forma iterativa, con cambios pequeños y feedback constante.

En el TP:

Fuimos construyendo por etapas (API → Docker → CI/CD → deploy)

Integramos cambios de forma continua (GitHub + PRs)

Lean

Busca eliminar desperdicio y hacer todo más eficiente.

En el TP:

Multi-stage Docker → imágenes más livianas

Automatización (CI/CD) → menos trabajo manual

Dockerfile			Docker compose	CI		CD	Monitoreo (new relic, Datadog,		
Dockerfile (requerido)	multi stage image	buenas prácticas	Buenas prácticas	Checks (build y unit test)	Flujo de mergeo	Publicacion dockerfile	Despliegue a render o un servicio	Dashboard (Hits, etc)	APM / Trazas
1	1	0,5	0,5	0,5	1	2	1	1,5	1

DOCKERFILE:

Dockerfile: funcional

Multistage: logrado con builder para compilación

Buenas practicas: new relic en dockerfile para desacoplar monitoreo del desarrollo, multistage y uso de imágenes livianas

DOCKER COMPOSE

Buenas practicas: uso de healthcheck, carga de variable de entorno en un .env

CI

Checks: ejecución de tests y buildeo en fase de CI para garantizar el correcto funcionamiento de la aplicación antes de integrar cambios

Flujo de mergeo: protección de rama main, si o si push a main vía pull requests de otra rama y tras ser aprobado por un tercero además de pasar los tests correspondientes

CD

Publicación dockerfile: publicación en docker hub durante la ejecución del cd

Despliegue a render: hook de render para desplegar una imagen en especifico salida de docker hub

MONITOREO:

Dashboard: visualización de métricas generales como la tasa de requests

APM/Trazas: monitoreo detallado de las requests, incluyendo tiempos de respuesta, errores, etc.